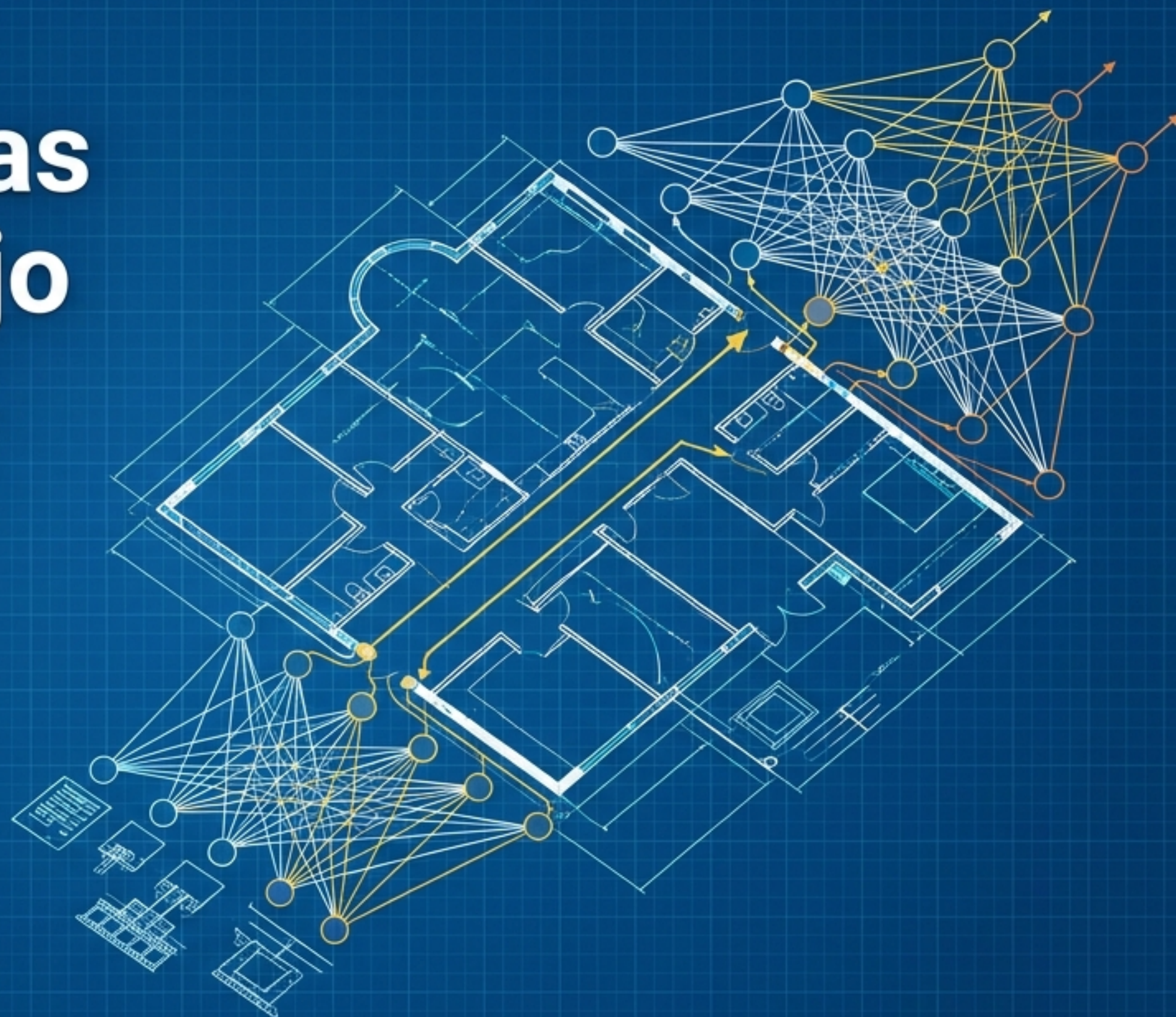


Deep Learning: Mejores Prácticas y Flujo de Trabajo

De los datos a las decisiones:
Guía práctica de arquitecturas
y evaluación

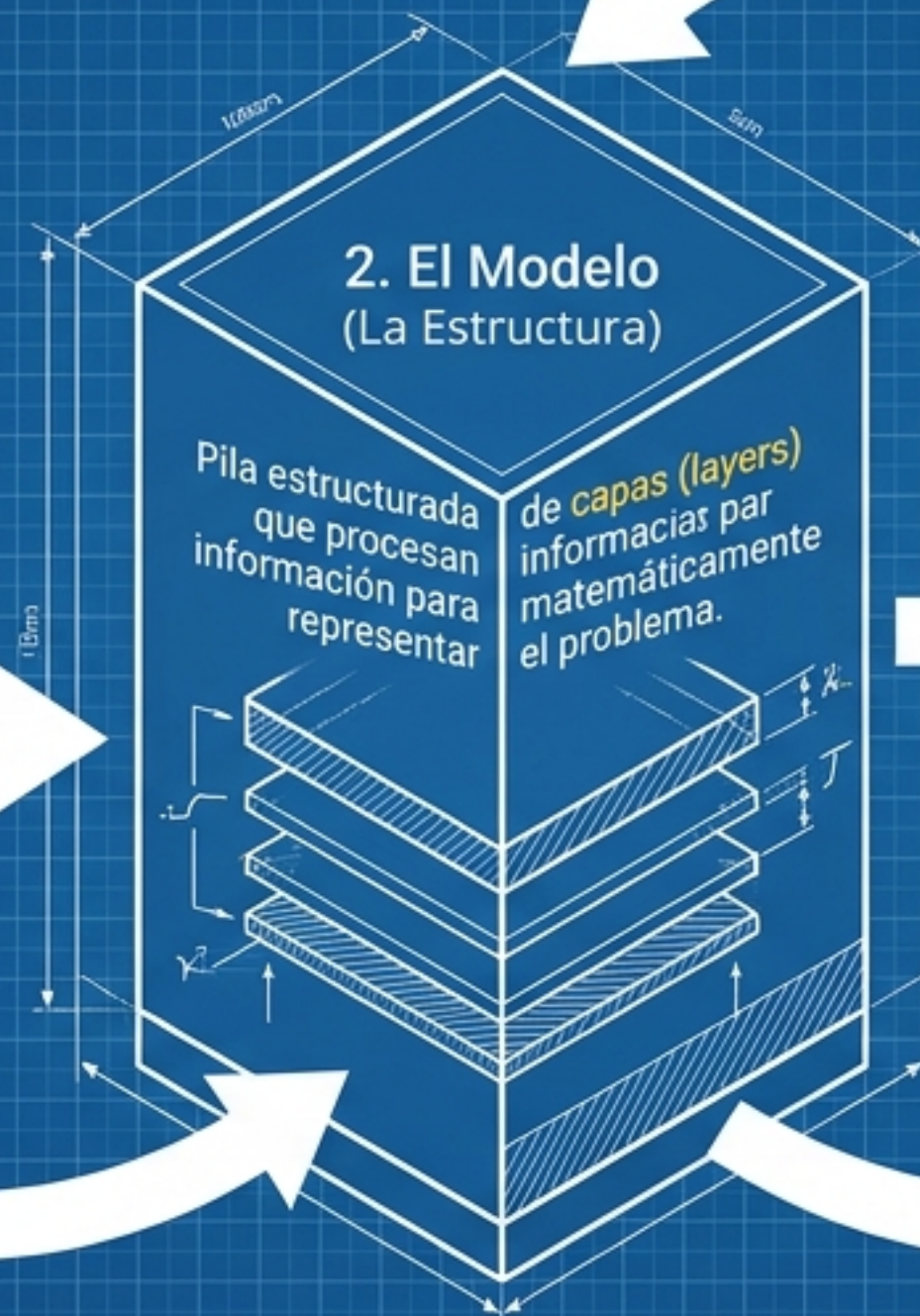


Materia:	Introducción Práctica al Deep Learning
Nivel:	Inicial - Intermedio

¿Qué es realmente el Deep Learning?

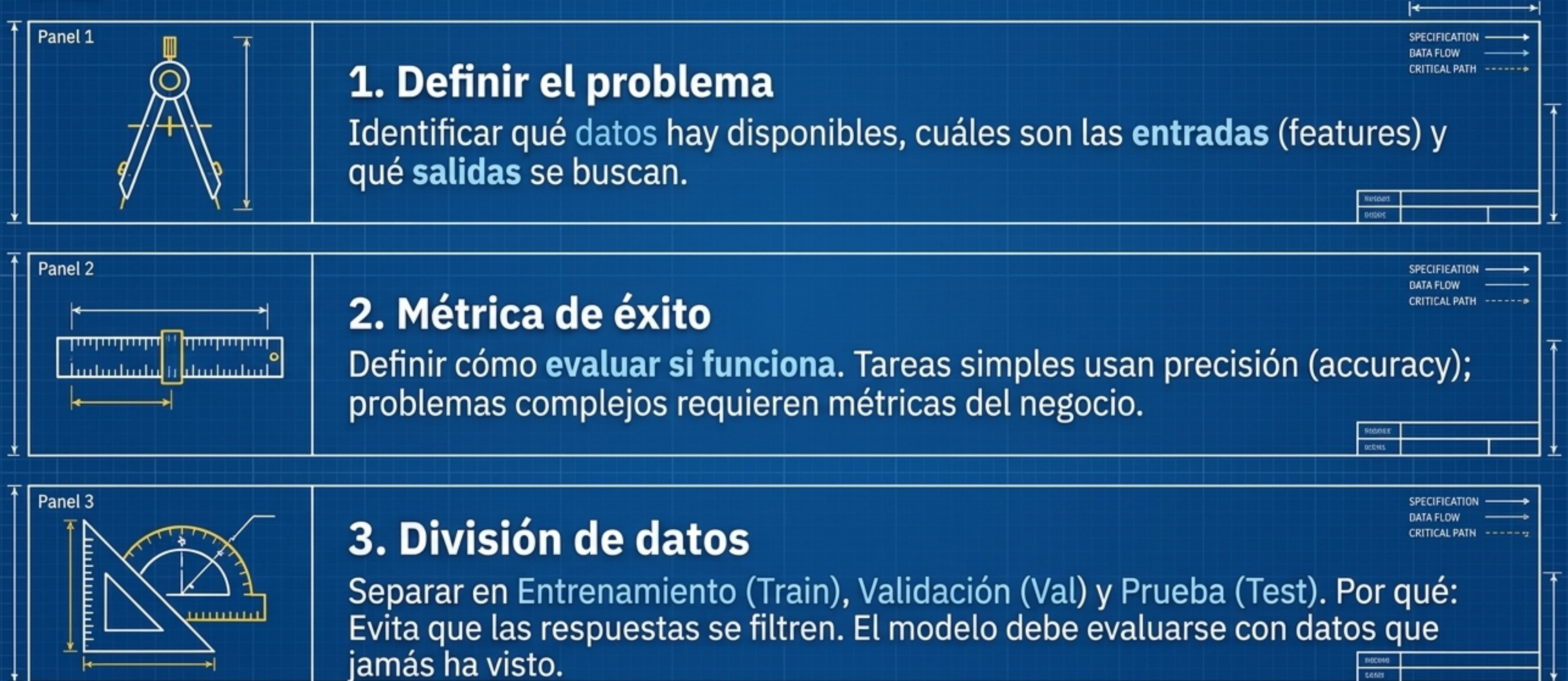
Specificación

El Deep Learning es un flujo de trabajo sistemático. Tomamos datos reales, los pasamos por capas matemáticas y dejamos que la computadora aprenda automáticamente a predecir resultados.



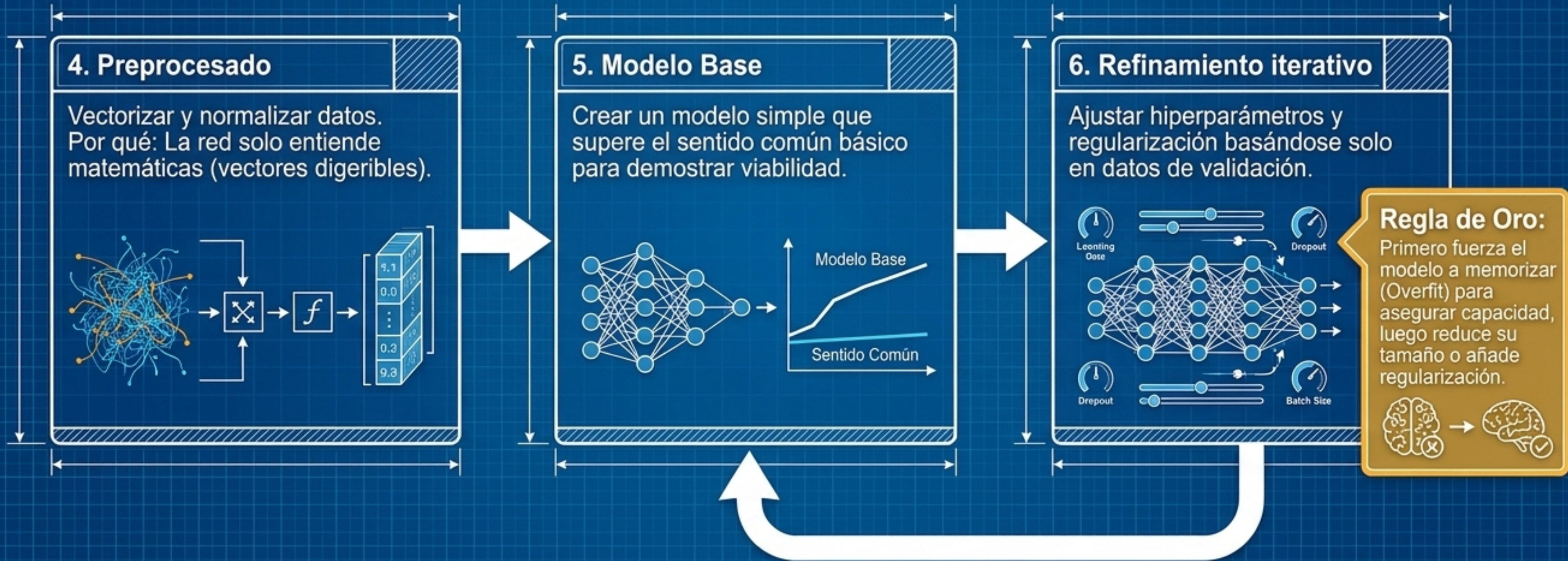
Metodología Paso a Paso: Diseñando el Problema

Antes de escribir código, estructuramos matemáticamente qué queremos lograr y cómo mediremos el éxito.



Metodología Paso a Paso: Del Dato al Modelo

Transformamos los datos, creamos una primera versión básica y la mejoramos de forma iterativa.



Redes Densas: Clasificación Binaria (Sí / No)

Arquitectura usada cuando el modelo decide entre dos únicas opciones (ej. Spam vs No Spam). Los objetivos (targets) son 0 o 1.

Specification Matrix	
Capa final	Una sola neurona (Dense de 1 unidad).
Activación	(sigmoid) Comprime la salida entre 0 y 1. Por qué: Se interpreta directamente como una probabilidad.
Pérdida	(binary_crossentropy) Mide el error entre nuestra probabilidad predicha y el valor real.

```
from keras import models
from keras import layers
model = models.Sequential()
model.add(layers.Dense(32,
activation='relu',
input_shape=(num_input_features,)))
model.add(layers.Dense(32,
activation='relu'))
model.add(layers.Dense(1,
activation='sigmoid'))
model.compile(optimizer='rmsprop',
loss='binary_crossentropy')
```

Redes Densas: Clasificación Multiclase de Etiqueta Única

Usada para clasificar un elemento en una sola categoría entre muchas posibles (ej. perro, gato o pájaro).

Specification Matrix	
Capa final	Tantas unidades como categorías totales (num_classes).
Activación	(softmax) Genera una distribución de probabilidades que suma 1 (ej. 70% gato, 20% perro, 10% pájaro).
Pérdida	(categorical_crossentropy) Mide error para formato one-hot (usar sparse_categorical_crossentropy para enteros).

```
model = models.Sequential()
model.add(layers.Dense(32,
activation='relu',
input_shape=(num_input_features,)))
model.add(layers.Dense(32,
activation='relu'))
model.add(layers.Dense(num_classes,
activation='softmax'))

model.compile(optimizer='rmsprop',
loss='categorical_crossentropy')
```

Redes Densas: Clasificación Multietiqueta

Una misma muestra puede pertenecer a varias clases a la vez (ej. una película puede ser Acción y Comedia). Targets codificados en k-hot.

Specification Matrix

Capa final	Igual al número total de clases (num_classes).
Activación	(sigmoid) Cada neurona de salida tiene su activación independiente (0 a 1). Por qué: Que sea Acción no resta probabilidad de que sea Comedia.
Pérdida	(binary_crossentropy) Se evalúa como si fueran múltiples clasificaciones binarias simultáneas.

```
model = models.Sequential()

model.add(layers.Dense(32, activation='relu',
                      input_shape=(num_input_features,)))

model.add(layers.Dense(32, activation='relu'))

model.add(layers.Dense(num_classes,
                      activation='sigmoid'))

model.compile(optimizer='rmsprop',
              loss='binary_crossentropy')
```

Redes Densas: Regresión (Valores Continuos)

No predecimos categorías, sino un valor numérico continuo (ej. predecir el precio exacto de una casa).

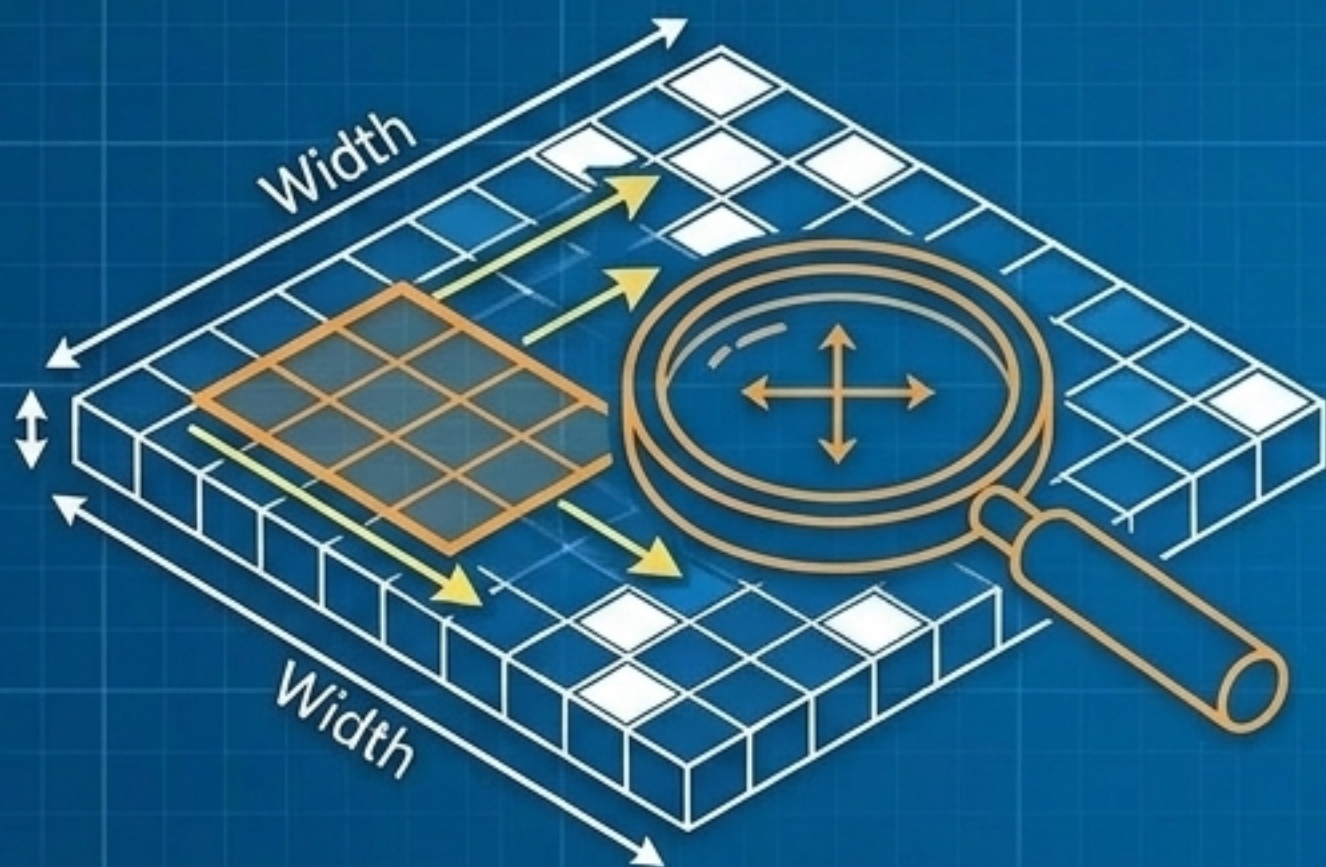
Specification Matrix	
Capa final	Unidades equivalentes a los valores a predecir (generalmente 1).
Activación	(Ninguna) No hay función de activación. Por qué: Queremos que la red devuelva cualquier número real sin limitarlo.
Pérdida	(mse o mae) Mean Squared Error (MSE) o Mean Absolute Error (MAE) miden la distancia numérica entre la predicción y el valor real.

```
model = models.Sequential()
model.add(layers.Dense(32, activation='relu',
                        input_shape=(num_input_features,)))
model.add(layers.Dense(32, activation='relu'))
model.add(layers.Dense(num_values))

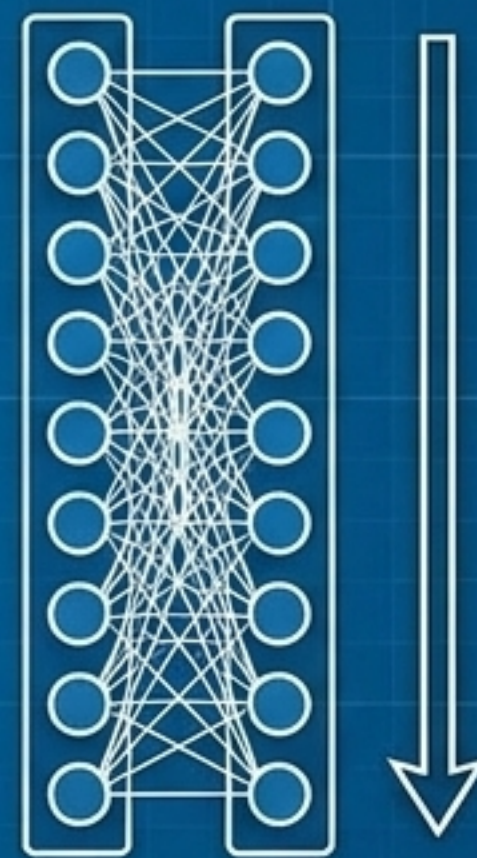
model.compile(optimizer='rmsprop', loss='mse')
```

Visión Artificial con CNNs: Procesando Imágenes

Las redes densas tratan los datos como listas planas. Las imágenes tienen estructura 2D (ancho y alto). Las CNNs conservan esta forma espacial.



CNN: Conserva topología espacial.
Usa "filtros" como lupas para escanear.



Dense: Destruye la estructura espacial,
tratando la imagen como una lista plana.

¿Qué aprende un filtro? Conceptos. En capas iniciales detecta bordes; en capas profundas aprende formas complejas (un ojo o una rueda).

La Magia del Pooling: Por qué menos es más

El Pooling (ej. MaxPooling) **reduce espacialmente la imagen** entre capas convolucionales, extrayendo el píxel más relevante. Evita que la dimensionalidad explote.

Nivel Calle: Mucho detalle fino, pero sin contexto de la ciudad.



Pooling
(Reducción Dimensional)



Nivel Satélite: Pierdes el detalle fino, pero la capa ahora ve la ciudad completa.

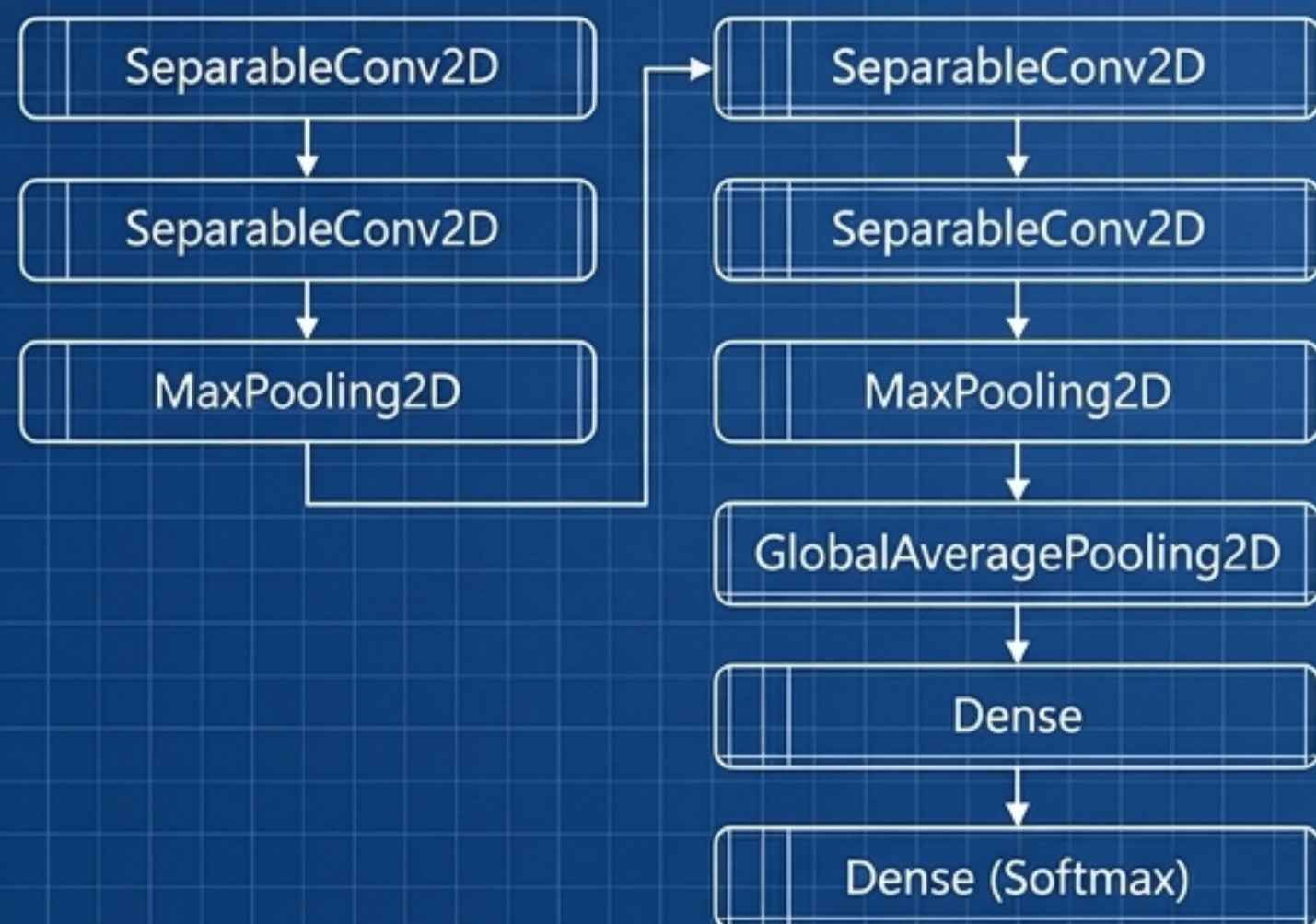


Por qué es fundamental: Reduce el tamaño espacial para permitir que capas posteriores vean una mayor extensión espacial de los inputs.

Ensamblando una CNN Moderna

Las CNNs intercalan **Conv2D** y **MaxPooling2D**. Al final, usan **GlobalAveragePooling2D** y capas **Dense** para clasificar.

La evolución (**SeparableConv2D**): Reemplazo directo de **Conv2D**. Es más rápida, representacionalmente eficiente, crea redes más ligeras y mejora el rendimiento.



```
model = models.Sequential()
model.add(layers.SeparableConv2D(32, 3,
    activation='relu', input_shape=(height, width,
    channels)))
model.add(layers.SeparableConv2D(64, 3,
    activation='relu'))
model.add(layers.MaxPooling2D(2))
model.add(layers.SeparableConv2D(64, 3,
    activation='relu'))
model.add(layers.SeparableConv2D(128, 3,
    activation='relu'))
model.add(layers.MaxPooling2D(2))
model.add(layers.SeparableConv2D(64, 3,
    activation='relu'))
model.add(layers.SeparableConv2D(128, 3,
    activation='relu'))
model.add(layers.GlobalAveragePooling2D())
model.add(layers.Dense(32, activation='relu'))
model.add(layers.Dense(num_classes,
    activation='softmax'))

model.compile(optimizer='rmsprop',
    loss='categorical_crossentropy')
```

Abriendo la Caja Negra del Deep Learning

Las redes profundas suelen ser criticadas por ocultar su conocimiento. Sin embargo, las CNNs tienen representaciones altamente visualizables. Necesitamos entender por qué el modelo toma ciertas decisiones.

Visualizando la Arquitectura

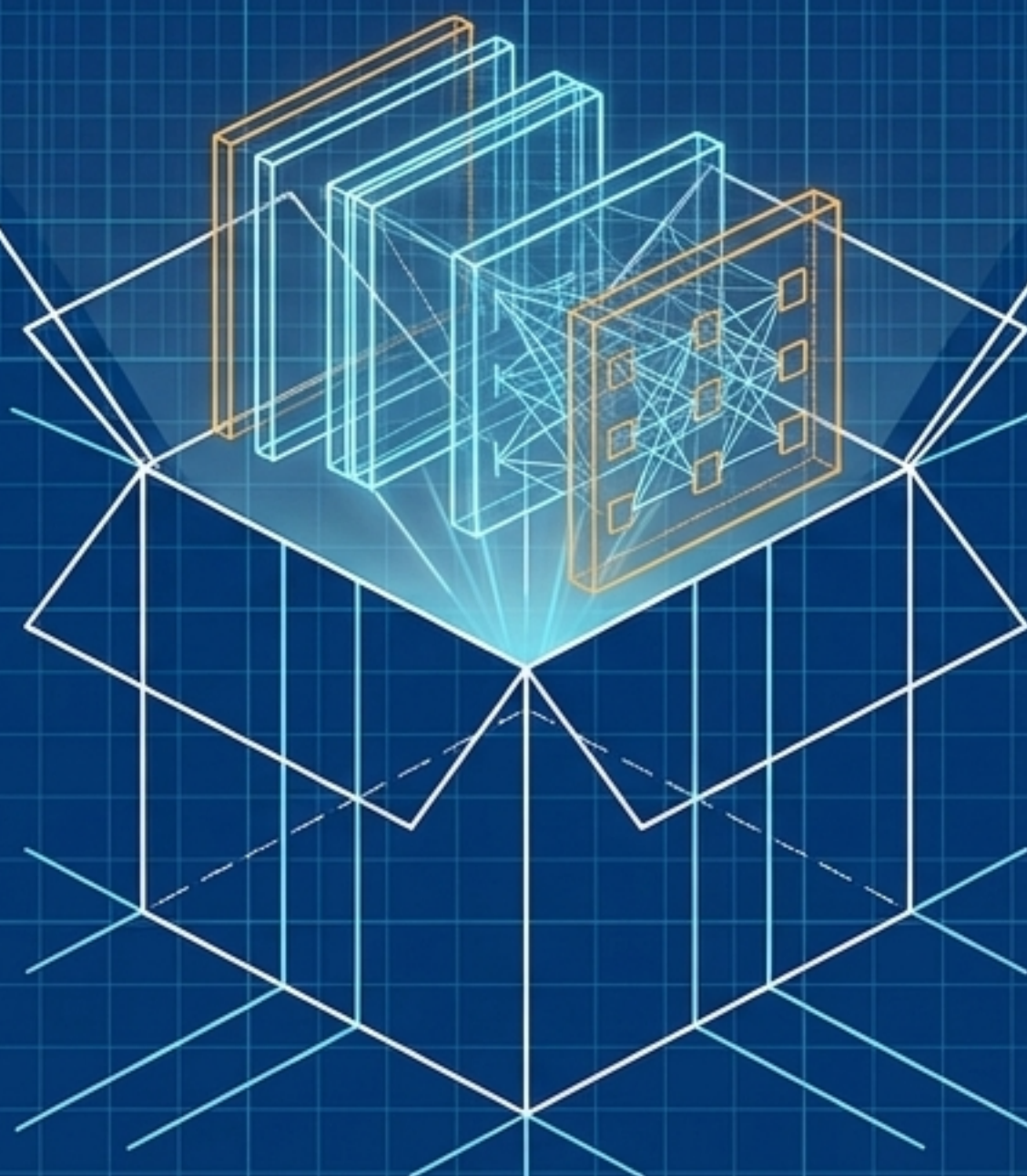
Para entender la estructura y dimensiones de volúmenes usamos:

```
model.summary()
```

para descripciones en texto.

```
plot_model
```

para generar figuras visuales.



Activaciones Intermedias

Ayudan a entender cómo **sucesivas capas** de la CNN **transforman** su input paso a paso de forma visual.

Entendiendo qué aprenden nuestras Redes

Inspeccionamos el modelo buscando el patrón exacto que hace reaccionar a un filtro específico.



Activation Maximization	Librería keras.vis (visualize_activation)
Genera artificialmente la imagen que maximiza la respuesta de un filtro. Muestra a qué patrón responde cada filtro.	Sirve para: - Filtros convolucionales - Capas Densas Categóricas - Capas Densas de Regresión
Fórmula Base: <code>input_img += gradient(activation, model.input)</code>	

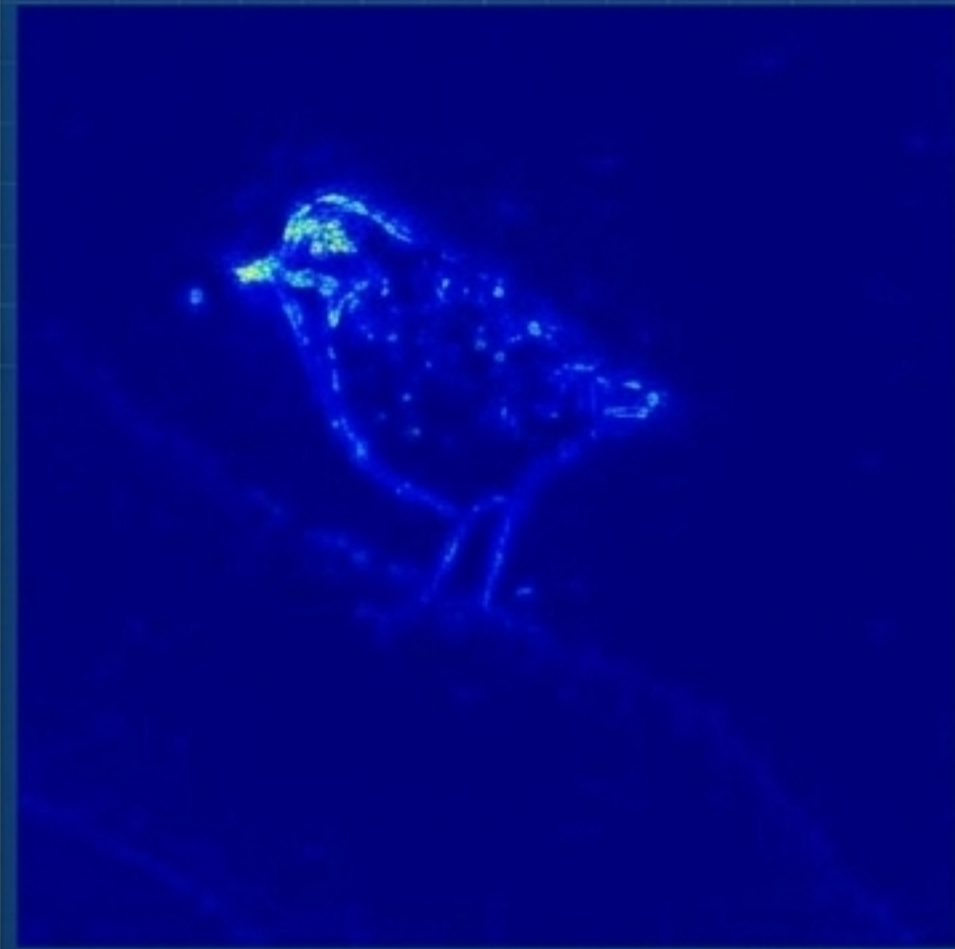
¿Dónde está mirando el modelo? (CAM)

Para localizar objetos y validar empíricamente la decisión de la red, vemos qué partes de la imagen impulsaron la clasificación final.

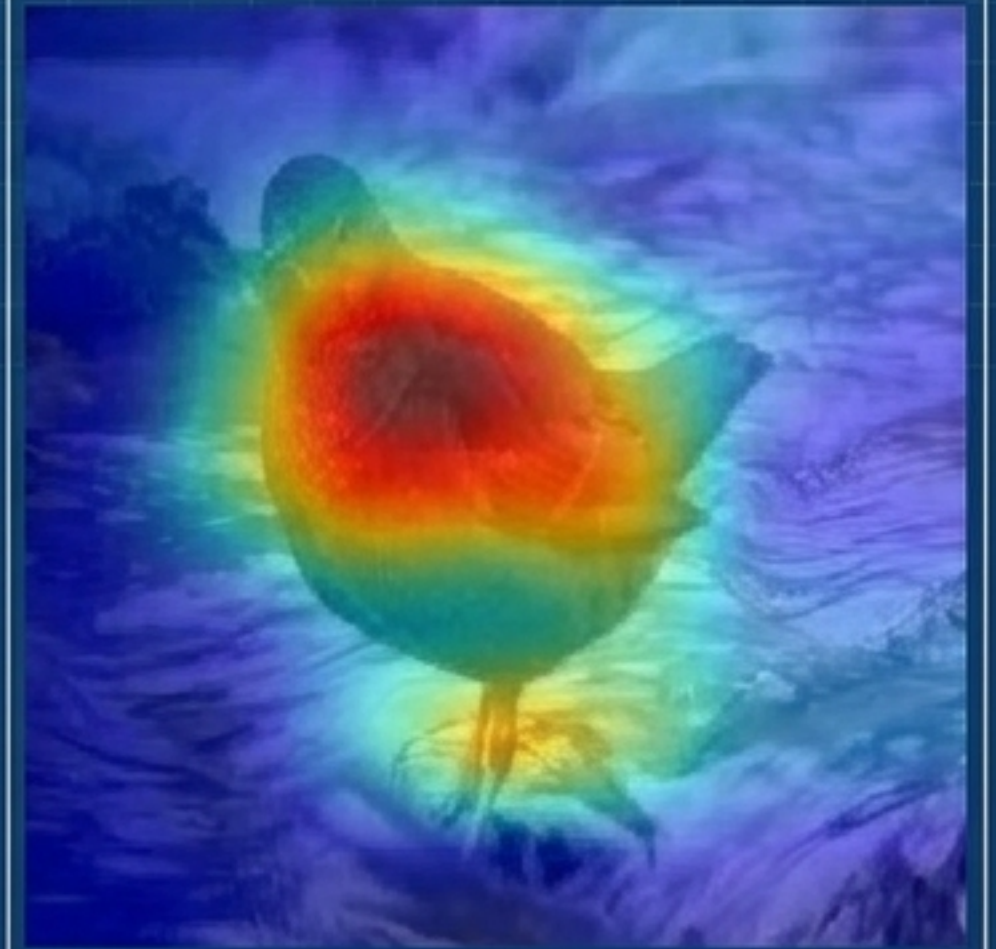
Revelation Process



1. Imagen Original



2. Bordes Relevantes
(visualize_saliency)



3. Mapa de Calor CAM
(visualize_cam)

Mecanismo interno: `compute gradient(output category, last_conv_layer.output)`

Resumen de Mejores Prácticas

Construir modelos robustos requiere metodología rigurosa, no depender de la suerte, y alinear las decisiones técnicas con la naturaleza del dato.

